3D Vision — Lab 5: Structure from Motion

Student Names	Student IDs	Group

1. Fundamental Matrix Estimation

Estimate the fundamental matrix for the three view pairs (1–2, 1–3, 2–3).

1.1 SIFT matching and robust F estimation

Paste your code for the filter_matches function.

Code — filter_matches

[Q1a] Explain the procedure followed to robustly estimate the fundamental matrix F. How did you detect keypoints? How did you match them? Why did you filter matches based on distance?

[Q1b] Explain the RANSAC approach used to find F. Which geometric distance is used to classify a match as an inlier — does it measure the distance between two points, or the distance from a point to a line? Justify your answer.

Paste your inlier match visualizations for all three view pairs (1–2, 1–3, 2–3).

Images — inlier matches 1–2, 1–3, 2–3

[Q1c] Report the number of inliers and the inlier ratio (inliers / total matches) for each pair (1–2, 1–3, 2–3). Which pair has the most/fewest inliers and why might that be?

2. Self-Calibration via Vanishing Points (K)

2.1 Intrinsic matrix estimation

Paste your code for computing K (the estimation of α and the construction of the K matrix).

Code — K estimation

[Q2a] What is a vanishing point? Why are vanishing points useful for camera self-calibration in man-made environments?

[Q2b] Report the intrinsic matrix K you obtained. What is the estimated focal length α?

K matrix and α value

3. Initial Two-View Reconstruction (P_1, P_2 and 3D points X)

3.1 Camera matrices (P_1, P_2)

Paste your code for computing E_{12} and selecting the correct P_2.

Code — E_{12} and P_2

Paste the numerical values of the camera projection matrices P_1 and P_2 you obtained.

P_1 (3×4) and P_2 (3×4)

[Q3a] Explain step-by-step how P_1, E_{12}, and P_2 were computed. Why does the decomposition of E give 4 candidate solutions? How do you select the correct P_2 among the 4 candidates?

3.2 Triangulation (3D points X from views 1 and 2)

Paste your code for the triangulation step (the DLT call and the reprojection-error / depth filter on the resulting cloud).

Code — triangulation + filtering

Paste the 3D point cloud plot (views 1–2 reconstruction with the two cameras).

Image — 3D point cloud (views 1–2)

[Q3b] Explain what triangulation does in this context (input, output, geometric idea). Why is a reprojection-error filter applied after the linear DLT? Which threshold (or rule) did you use?

[Q3c] Evaluate qualitatively the reconstruction. Is it geometrically plausible? Can you identify objects or scene structures in the 3D point cloud? Are there any outliers?

3.3 Reprojection error of the 1–2 cloud

Paste the reprojection-error histogram you obtained (view 1 and view 2 side by side).

Image — reprojection-error histogram (views 1 and 2)

[Q3d] Report the mean and median reprojection error in views 1 and 2. Is the distribution similar between the two views? Does it make sense to you?

4. Adding the Third Camera (P_3)

4.1 2D–3D correspondences ($X \leftrightarrow x_3$)

Paste your code for the intersection matching loop (finding common keypoints across views 1–2 and 1–3).

Code — intersection matching

[Q4a] Explain the purpose of computing the intersection between the (1–2) and (1–3) match sets. Why do we need 2D–3D correspondences to add a new camera view?

4.2 Resectioning (P_3)

Paste your code for the single_resectioning function.

Code — single_resectioning

Paste the numerical value of the projection matrix P_3 you obtained.

P_3 (3×4)

[Q4c] Explain the resectioning technique step by step.

[Q4d] How many 2D–3D correspondences are required for resectioning? Justify by relating to the degrees of freedom of the projection matrix P.

[Q4e] Why is RANSAC applied on top of the linear resectioning?

[Q4c1] Explain the linear system you assembled inside single_resectioning. Why are there two rows per 2D–3D correspondence (instead of one or three)? What does the right-singular vector associated with the smallest singular value of A represent, and why do you multiply by T_1^{-1} and T_2 at the end?

[Q4c2] In find_resectioning_inliers, which residual do you compare against the threshold th — pixel error in the image or 3D error in space? Why? How would you pick a sensible value of th for this dataset?

[Q4c3] Inside ransac_resectioning_adaptive_loop, what is the minimum sample size s and why exactly that number? How does the adaptive update of N (the iteration budget) work each time a larger inlier set is found? Why do you re-fit single_resectioning on the full inlier set at the end?

4.3 Camera decomposition (K_3, R_3, t_3)

Paste the printed values of K_3, R_3, and t_3 obtained from decomposing P_3.

Matrix values — K_3, R_3, t_3

[Q4f] Is K_3 similar to the previously estimated K from Section 2? Should they be similar? Explain any differences you observe.

[Q4g] What can you interpret from R_3? How large is the rotation? In which coordinate frame is this rotation expressed?

[Q4h] What can you interpret from t_3? In which coordinate frame is this translation expressed?

4.4 Qualitative view (all cameras + 1–2 cloud)

Paste the 3-camera visualization plot (the three cameras together with the 1–2 cloud).

Image — 3-camera visualization

[Q4i] Do the camera positions confirm your interpretations of R_3 and t_3 from Section 4.3?

5. Cloud Densification and Evaluation

5.1 Densified cloud — union of 1–2 ∪ 1–3

Paste your code that triangulates the F_{13} inliers with P_1 and P_3, filters outliers, and merges with the 1–2 cloud.

Code — densification (views 1–3)

Paste the merged 3D point cloud (views 1–2 ∪ 1–3) with all three cameras.

Image — densified point cloud (union)

[Q5a] Why does adding the 1–3 triangulation increase the cloud density? Which guarantees that the new points are geometrically consistent with the already-recovered cameras (P_1, P_2, P_3)?

[Q5b] How many new 3D points did the 1–3 triangulation contribute (after the outlier filter)? Compare this number to the 1–2 cloud size.

5.2 Strict three-view intersection ($F_{12} \cap F_{13} \cap F_{23}$ inliers)

Paste your code for the triplet intersection loop (matches that survive in all three pairs).

Code — triplet intersection

Paste the strict-intersection point cloud with all three cameras.

Image — strict-intersection cloud

[Q5c] How many triplet matches survived the intersection across all three pairs? How many points remained after the outlier filter?

[Q5d] Compare the union variant (5.1) and the strict-intersection variant (5.2) in terms of cloud size and geometric reliability. When would you prefer each one?

5.3 Quantitative evaluation of the three clouds

Paste the numerical evaluation output: inlier ratios per pair + per-cloud reprojection-error statistics (mean / median / max).

Numerical evaluation — inlier ratios + reprojection errors of the three clouds

[Q5e] Evaluate qualitatively the three 3D representations you obtained (1–2 only, 1–2 ∪ 1–3, strict 1–2 ∩ 1–3 ∩ 2–3). Which scene elements can you recognize? Is anything poorly represented? Interpret why.

[Q5f] Pipeline improvement. Based on the inlier ratios, cloud sizes, and reprojection errors you reported above, how would you improve the pipeline to obtain denser point clouds AND lower reprojection errors? Discuss at least two concrete ideas. For each idea, name the metric you expect it to improve (density or error) and explain why.

6. VGGT — Visual Geometry Grounded Deep Structure from Motion

In this section you will explore a recent deep-learning approach to the Structure from Motion problem.

Visit the project page at https://vgg-t.github.io/ and try the interactive demo at https://huggingface.co/spaces/facebook/vggt.

[Q6a] Briefly explain how VGGT works and what its key innovation is. How does it differ from the classical incremental SfM pipeline you implemented in this lab (feature extraction → matching → RANSAC → triangulation → resectioning)?

[Q6b] Try the demo with your own images (use at least 2–3 photos of a scene of your choice). Paste your input images and the output reconstruction below. Comment on what worked well and any limitations you noticed.
Input images:

Images — your input photos

Output reconstruction and your comments:

Output reconstruction + comments

7. AI Usage Reflection

Which AI tools have you used?
(ChatGPT / Copilot / Gemini / other — list all)

For which tasks did you use them?
(debugging, code generation, understanding concepts, writing, other)

What did the tool succeed on? What did it fail on?

Have you changed how you use these tools compared to previous labs?

Paste a successful example: include the prompt you used and the output you obtained.

Successful prompt + output

Paste a failed example: include the prompt you used, the output obtained, and explain what went wrong.

Failed prompt + output

8. Self-Assessment

Provide an overall explanation of the lab.
Describe the goals of the lab clearly in your own words.

What have you learned?
Did the lab help you understand and visualize the incremental SfM pipeline?

Which parts of the notebook did you succeed in?
Describe the sections where you felt confident and explain why you think they were successful.

Which parts did you fail to solve?
Be honest about the areas where you faced difficulties. What challenges did you encounter? How would you approach these issues in the future?

Are there any concepts that are not completely clear yet?
Describe what is still confusing and what topic or explanation would help you.

Extras

Add any extra results, plots, or comments below — including the optional derivation from [Q2d]. If needed, use multiple pages for this section.